

OASYS Backend Component

OasysBackendState v1.3

Author: Eric J. FRANCOIS

August 5, 2026

1 description

The `OasysBackendState` class replaces standard PHP sessions for storing temporary data on the server side for a specific user. It stores data in a database table (`stateBackend`) instead of using the local file system. This mechanism is especially useful in load-balanced environments where local session files cannot easily be shared across instances.

Each user is uniquely identified by a `stateId` which is automatically generated and securely stored in a cookie (`oasysStateBackend`). This cookie is sent alongside each request, analogous to a standard session cookie.

2 usage

The class is implemented as a Singleton to ensure only one instance is active per request. Its properties are handled dynamically using PHP magic methods, meaning you can directly set or get any arbitrary property you need.

Example

Creating a new instance or getting one that already exists:

```
$state = OasysBackendState::getInstance();
```

Setting a dynamic property:

```
$state->myCustomProperty = $value;
```

Retrieving a dynamic property:

```
$value = $state->myCustomProperty;
```

Checking if the browser has an active state without initializing it:

```
$isActive = OasysBackendState::browserHasActiveState();
```

Dependencies:

This class depends on the `OasysSettings` class for its configuration, `rixPDO` for database interactions, and it may throw `StateExpiredException` or `InvalidKeyException` in certain edge cases.

3 dynamic properties

The `OasysBackendState` instance allows assigning values to dynamically named properties. When you assign or read a property, magic methods intercept the call and route it to the database where the values are serialized into a JSON column.

Besides getting and setting values, the default PHP commands of `isset()` and `unset()` are also supported for state properties.

4 instance methods

The following public methods are available on an `OasysBackendState` instance.

`getStateId()`

Returns the active `stateId` string (from the cookie). Generates and stores a new one if it doesn't exist.

<code>rotateStateId()</code>	Cycles the current <code>stateId</code> and issues a new cookie. Helps mitigate session fixation attacks.
<code>eraseState()</code>	Deletes the current state record entirely from the database and instructs the browser to delete the cookie.
<code>isStateActive()</code>	Checks the database to verify if the current instance's <code>stateId</code> hasn't exceeded the configured timeout.
<code>getAllStateData(\$unwrap = true)</code>	Returns an array or a raw JSON string of all data stored for the current <code>stateId</code> .
<code>nukeAllStatesExceptCurrent()</code>	Deletes all backend states in the database except for the current instance's <code>stateId</code> .
<code>backupState()</code>	Temporarily stores the current state data in memory (without unwrapping) to allow restoration later.
<code>restoreState()</code>	Inserts or overwrites the database record for the current <code>stateId</code> with the previously backed-up data.

4.1 getAllStateData parameters

<code>\$unwrap</code>	<i>bool</i>	Optional parameter, defaults to true. If false, returns the raw JSON string instead of an array.
-----------------------	-------------	--

* mandatory parameters

5 static methods

Static methods allow you to interact with the backend state system globally without directly instantiating the state data of the current user.

<code>getInstance()</code>	Returns the singleton instance of the class. If it doesn't exist, it is initialized and the timestamp is updated.
<code>browserHasActiveState()</code>	Checks if a cookie is present and if its corresponding <code>stateId</code> is still active in the database.
<code>isStateIdActive(\$stateId)</code>	Verifies if a specific <code>stateId</code> provided in the arguments is active in the database.
<code>getNumberOfActiveClients()</code>	Returns the count of all active states in the database, or -1 if the count cannot be determined. A custom timeout (in minutes) can be passed.
<code>fetchAllActiveStates()</code>	Returns an array of all active states, including their data, modification date, and inactivity time.

5.1 isStateIdActive parameters

<code>\$stateId*</code>	<i>string</i>	The specific 10-byte hex <code>stateId</code> to check against the database.
-------------------------	---------------	--

* mandatory parameters

5.2 `getNumberOfActiveClients` parameters

<code>\$timeout</code>	<i>int</i>	An optional inactivity limit in minutes. If omitted, it defaults to the <code>backendInactivityTimeout</code> setting.
------------------------	------------	--

* mandatory parameters